Security Assessment Report

ScanDate: 18Nov2025
Tool: SolidityScan
Scope: TroyvestPresale.sol
Assessment Method: Automated static analysis

01. Executive Summary

The Troyvest Presale contract was scanned to detect security, logic, and gas-related weaknesses.

Overall Security Score: 93.48 / 100

Total Issues Detected: 38

None are security-critical.
All findings are **Low**,**Informational** ,or **Gas optimizations**.

Severity Breakdown

Severity	Count	Notes
Critical	0 0 0 3	Norisk of fundloss None
High	7	None Minor logic/style
Medium		issues Missing
Low		documentation
Informational		Efficiency only
Gas Optimization	28	

Conclusion:
The contract is **functionally safe**,but requires **clean-up and optimization** for gas and clarity.

02. Vulnerability Classification

Issues grouped by category:

- **Low severity** → Minor correctness or style
- **Informational** → Documentation/NatSpec
- **Gas** → Cheaper operations, memory usage, increment ops, type optimizations

---

03. Findings Summary

Low Severity Issues (3)

ID	Description	Line
L001	Incorrect variable name case	93
L002	Missing require checks inside constructor	99
L003	Missing error messages for require	109

Informational Issues (7)

ID	Description	Lines
INFO001	Missing `@notice` for contract	5 17,
INFO002	Missing `@notice` on functions	120, 333
INFO003	Missing NatSpec params	88–105
INFO004	Return variable not documented	322
INFO005	Return variable name mismatch	333
INFO006	Missing @param for view functions	366

INFO007 Missing struct NatSpec 379

Gas Optimization Issues (28)

Grouped by type:

GO05 — Cheaper Inequalities in require()

- Use `<` instead of `<=` when safe.
  Lines: 284, 314

GO06 — Default int values manually reset (3 instances)

- Writing `x = 0` wastes gas.
  Lines: 178, 189, 461

GO07 — Constructor should be payable (1 instance)

- Saves deployment gas.
  Lines: 88–140

GO08 — Reverting functions can be payable (10 instances)

- Marking pure/view functions as payable reduces runtime cost.
  Lines: 144, 156, 166, 172, 177, 207, 241, 295, 327, 337

GO09 — Function should return struct (4 instances)

- Returning many values costs more gas than a struct.
  Lines: 179–128, 349–373, 442–473, 498–525

GO10 — Gas optimization for state variables (1 instance)

- Prefer `s = s + 1` over `s += 1` in storage.
  Line: 190

GO11 — Increment using ++i instead of i++ (1 instance)

Line: 519

GO12 — Smaller data types cost more (6 instances)

- Replace `uint8/uint16/uint32` with `uint256`.
  Lines: 212, 214, 229, 231, 292, 322

GO13 — Storage variable caching (8 instances)

Avoid repeated sload.

Lines: 219–233,237–245,247–255,273–304,307–329,332–329, 498–525

GO14 — Storing storage variable in memory (8 instances)

Lines: 198, 281, 308, 363, 387, 409, 428, 454

---

04. Detailed Description of Key Issues

Low Severity

1. Missing Validation
Constructor missing input validation may allow bad config if wrong values passed.
2. Unclear Error Messages
Require statements without messages reduce traceability.

---

05. Security Impact Assessment

✓ No critical security risk

No overflow, no reentrancy, no unprotected state change, no unchecked external calls.

✓ Presale logic is structurally safe

BNB/NATIVE and stablecoin handling safe as long as caller handles decimals correctly.

✓ Gas issues do not break functionality

All are optional but recommended.

---

06. Recommendations (Action List)

High-priority (Safe Launch Essentials)

- Add **require() error messages**
- Validate all constructor params
- Fix variable naming consistency

Medium-priority (Gas Savings 5–15%)

- Cache storage variables locally
- Replace small integer types with uint256
- Remove resetting to zero (delete instead)
- Use ++i patterns
- Make constructor payable

Low-priority (Clean Code)

- Add full NatSpec comments
- Replace multi-return with `struct`
- Simplify comparisons in require()

---

07. Final Verdict

Deployment Status:

🟢

Safe to deploy after applying cleanup + gas improvements
Novulnerabilitiesthatcancauselossoffunds.

Risk Level:

🟢 **Low Risk**

Contract Strength:

- Strong logic
- Modular
- Low attack surface
- Mostly optimization gaps